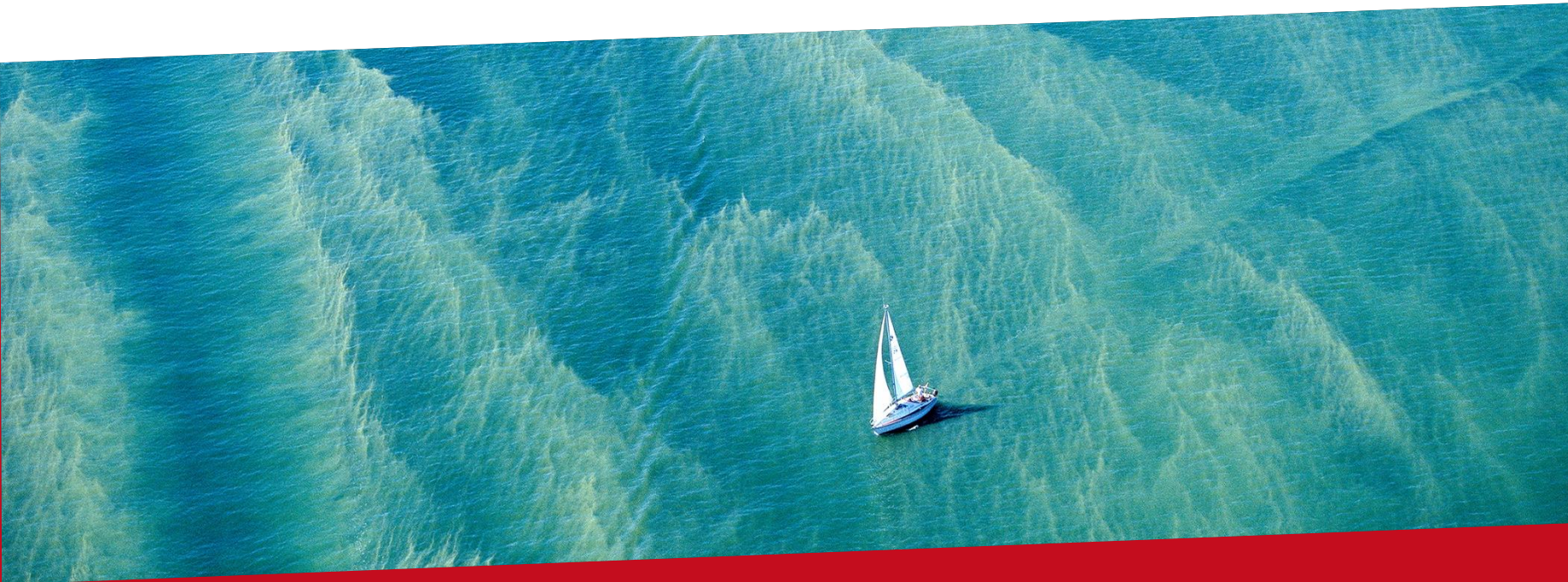




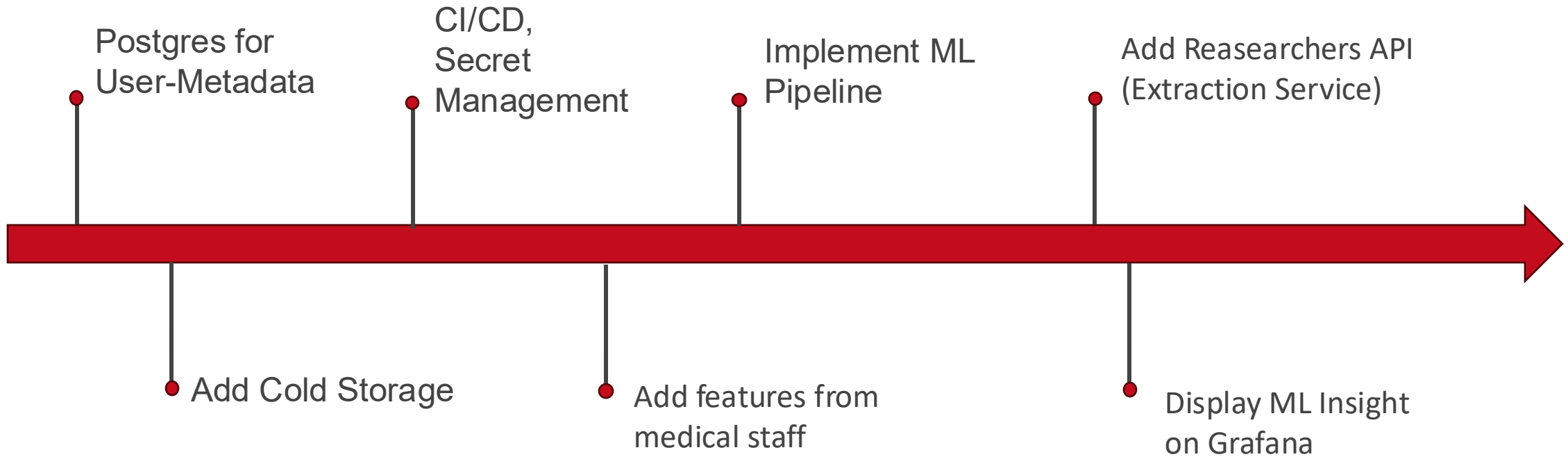
Roadmap

Current State of the System

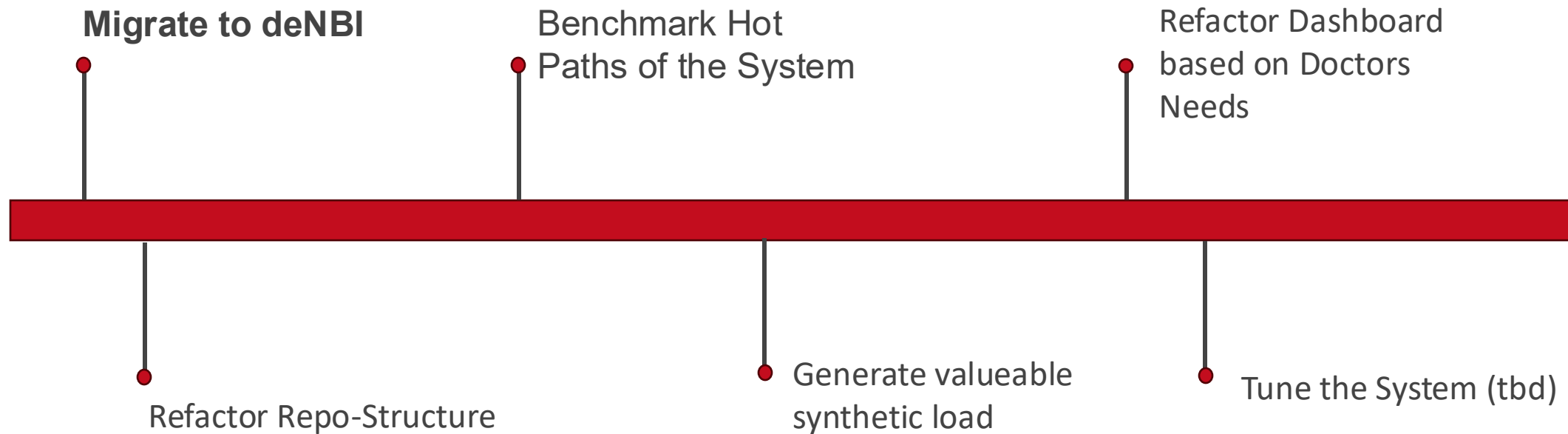
Our system is running in a temporary (single node) cluster on Hetzner

- Running App (connects watch to the system)
- Running Ingestion Service (HTTP-API), Running Mosquitto-Broker (MQTT-API)
- Running (Single Node) Kafka
- Running Kafka-Streams Mapping-Service (declarative)
- Running InfluxDB (+Telegraf)
- Running Grafana Server + (First-Draft) Dashboard

Next Steps: Features



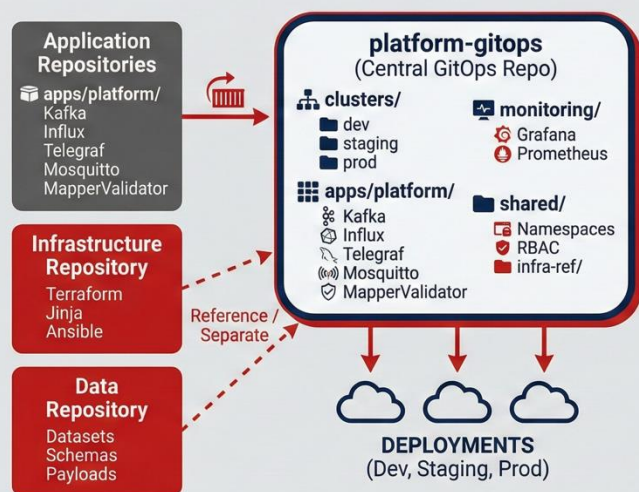
Next Steps: Refinements



Repository Landscape Comparison

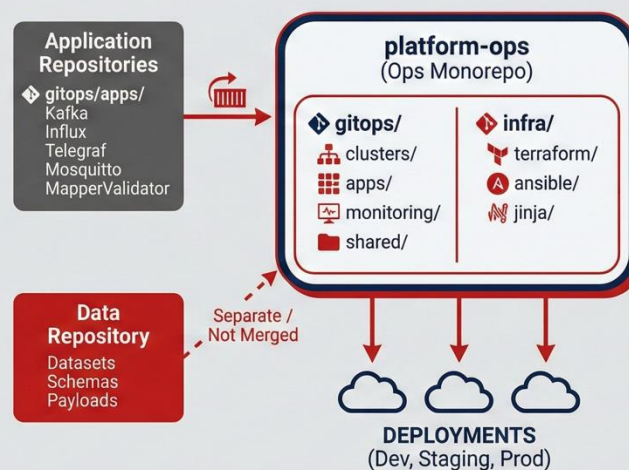
REPOSITORY ARCHITECTURES COMPARISON: CLEAN SEPARATION VS. OPS MONOREPO

VARIANT A: CLEAN SEPARATION



Clear separation. Infra & Apps distinct, referenced by central GitOps repo.

VARIANT B: OPS MONOREPO



Consolidated Ops repo. GitOps & Infra code coexist, separate App & Data repos.

Both variants achieve GitOps. Variant A prioritizes strict separation; Variant B offers unified operational view.

Variant A: Separate Repositories

Uses distinct repositories for configs, apps, infrastructure, and data. This model promotes clear ownership and low coupling, supporting stable CI/CD processes.

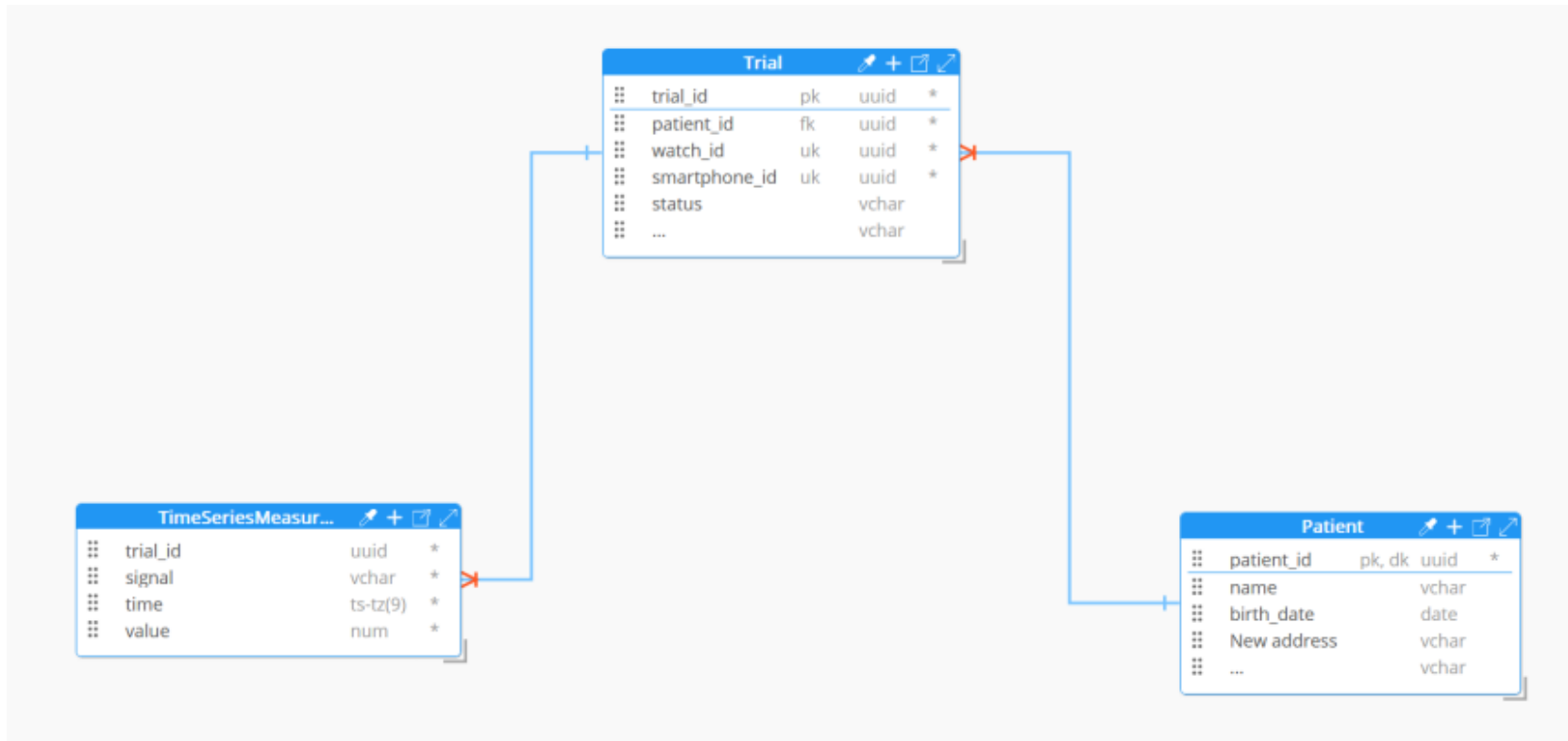
Variant B: Merged Operations Repo

GitOps and infrastructure are combined in a single platform-ops repository, with apps and data remaining separate. This gives a unified operational perspective but adds complexity.

Trade-offs and Implications

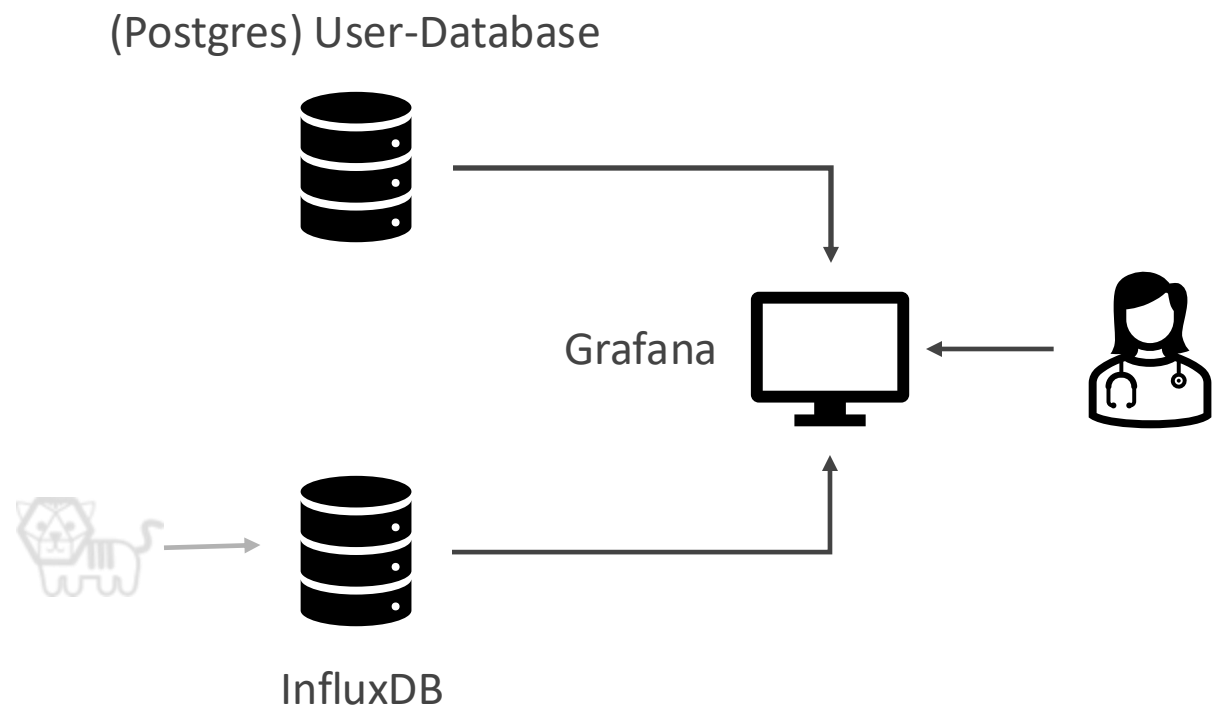
Variant A offers simplicity and stability, while Variant B improves oversight but increases coupling and maintenance challenges.

Data Model



measurement, id=<trialID>, tag2=val2 field1="v1", field2=1i 00000000000000000000

Data Model II: Combination of User-Data and Measurements



To enable a **Drilled-Down Population View**:

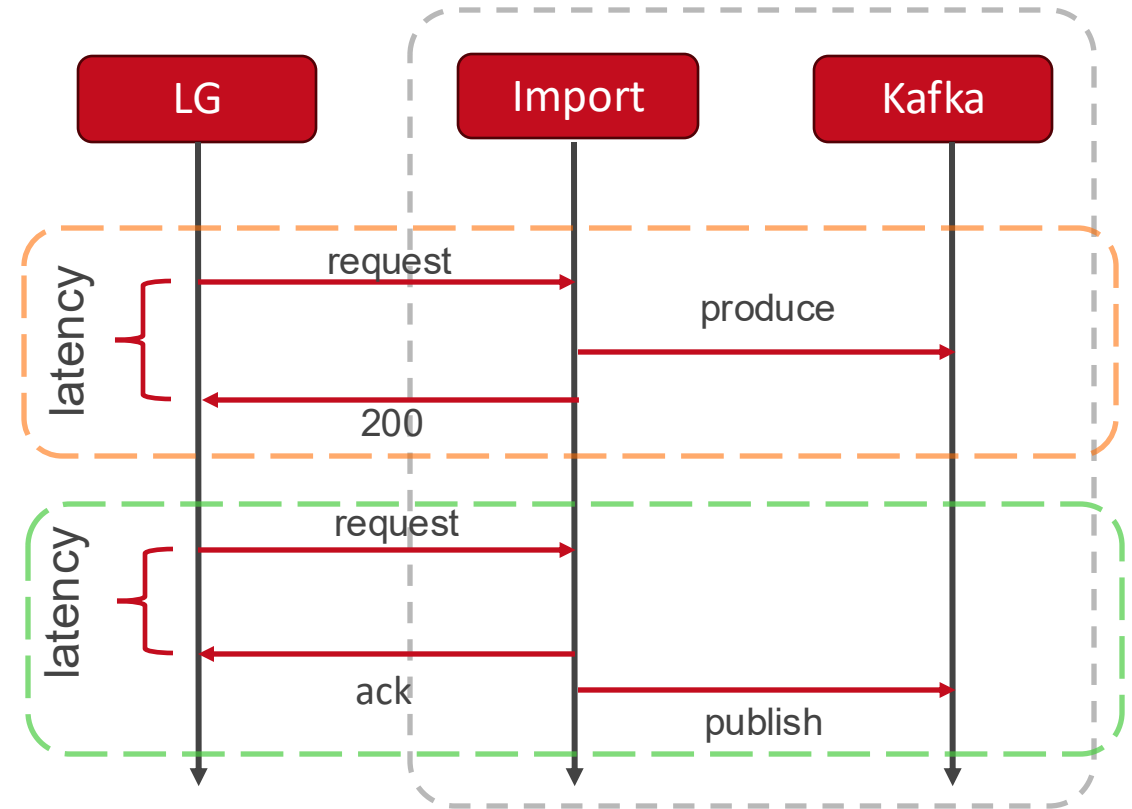
1. Grafana queries User-Database
2. Grafana forms groups based on attributes
3. Grafana extracts TrailIDs for all group members
4. Grafana queries InfluxDB using those TrailIDs
5. Grafana displays aggregated or detailed measurements

Load Generator: Client-Centric

wplug has 2 different purposes

1. Testing the System, by writing „synthetic“ messages to the system
2. Evaluate Client-Centric Latency
3. Trace Process-Time

wplug should run both on local machines and as a node in the cluster



Misc: Benchmarking with our Load Generator

